

08 - Implementation Roadmap

Project: Dragon Merchant Guild (*Working Title*)

Document Version: 1.0

Status: Living Development Roadmap

Purpose

This roadmap translates the game's vision into a sequence of achievable development milestones.

It is intentionally aligned with the C# learning curriculum.

The project exists primarily as a vehicle for becoming a better software engineer.

Every milestone should introduce only the concepts currently understood by the developer.

Future concepts should not be implemented early simply because they are known to exist.

The game should evolve naturally alongside the developer's knowledge.

Core Philosophy

Build the simplest version first.

Then improve it.

Do not skip ahead.

The goal is not to build the perfect merchant simulator immediately.

The goal is to finish many small systems that gradually become more sophisticated.

Development Principles

Throughout development:

- ✓ Keep the game playable.
 - ✓ Finish features before expanding them.
 - ✓ Refactor only after understanding improves.
 - ✓ Favor readability over cleverness.
 - ✓ Celebrate working software.
-

Milestone 0 — Project Foundation

Curriculum

Before Module 1

Goal

Prepare the project.

Tasks

- Create solution
 - Create Git repository
 - Configure folder structure
 - Create README
 - Create project journal
 - Create assets folder
 - Create documentation folder
-

Deliverable

A clean, organized project ready for development.

Milestone 1 — Console Prototype

Curriculum

Goal

Create a playable console game.

No graphics.

No save system.

Focus entirely on programming fundamentals.

Features

- Main menu
 - Exit
 - Create guild
 - Display guild information
 - Display dragons
 - Display workers
 - Pass one day
 - Basic logging
-

Dragons

Initially only:

- Stonejaw
- Hearthscale
- Riverveil

Each dragon only stores:

- Name
 - Hunger
 - Stamina
-

Contracts

Simple text contracts.

Example:

Mine 20 Stone.

Reward:

50 Gold

Resources

Initially:

- Gold
 - Stone
 - Food
-

Deliverable

A playable console loop.

Milestone 2 — OOP Expansion

Curriculum

Module 2

Goal

Transform the procedural prototype into a proper object-oriented application.

Introduce

- Dragon class hierarchy
 - Worker classes
 - Contract classes
 - Buildings
 - Guild class
-

New Dragons

Add:

- Mosshorn
 - Longtail Courier
-

New Systems

- Dragon care
 - Building construction
 - Worker assignments
-

Deliverable

Every major game object is represented by an appropriate class.

Milestone 3 — Collections

Curriculum

Collections & Generics

Goal

Replace fixed values with collections.

Introduce

- Dragon roster
 - Worker roster
 - Contract board
 - Building list
 - Inventory
-

New Systems

- Dragon adoption
 - Hiring workers
 - Multiple active contracts
 - Resource storage
-

Deliverable

The guild can grow beyond predetermined limits.

Milestone 4 — LINQ

Curriculum

LINQ

Goal

Improve data management.

Examples

Find:

- Hungry dragons
- Idle workers
- Highest-value contracts
- Low food supply
- Tired dragons

Sort:

- Reputation
 - Dragon age
 - Gold value
 - Worker experience
-

Deliverable

Cleaner, more expressive gameplay logic.

Milestone 5 — Persistence

Curriculum

File I/O

Goal

Save progress.

Features

- Save
 - Load
 - Autosave
 - Multiple guilds
-

Deliverable

Persistent gameplay.

Milestone 6 — Events

Curriculum

Intermediate C#

Add

- Weather
- Merchant visits
- Guild inspections

- Festivals
 - Dragon illness
 - Random discoveries
-

Deliverable

Every day feels different.

Milestone 7 — Async

Curriculum

Async & Await

Introduce

Background activities.

Examples:

- Mining expeditions
- Courier deliveries
- Construction
- Research

The player can continue playing while long-running activities progress.

Deliverable

A living guild.

Milestone 8 — User Interface

Replace console menus.

Possible future targets:

- Avalonia

- Godot
- Unity
- Web UI

The underlying game logic should require minimal changes.

Milestone 9 — Polish

Examples:

- Better balancing
 - Sound
 - Animations
 - Save optimization
 - Better AI
 - Expanded economy
-

Things Intentionally Delayed

The following systems should not be implemented until later unless required by learning.

- Multiplayer
- Procedural world generation
- Multiplayer economy
- Dragon breeding
- Genetics
- Combat
- Legendary dragons
- Political simulation
- Dynamic markets
- Multiplayer guilds

Simple systems are easier to understand, debug, and improve.

Definition of Success

The project is successful if:

- Every curriculum module resulted in meaningful game improvements.
- The codebase remains understandable.
- The developer can explain every major system.

- The game is enjoyable to work on.
 - The project reaches a playable state without relying on AI-written production code.
-

Final Reminder

Never compare this project to commercial games.

This game has a different purpose.

Its greatest achievement will not be the number of dragons, buildings, or mechanics it contains.

Its greatest achievement will be transforming its developer into an engineer capable of building worlds far larger than this one.

Every feature completed is another lesson learned.

Every bug fixed is another step forward.

And every dragon added to the guild is another reminder that great software—like great partnerships—is built patiently, one day at a time.